

Secure Guide

AWS 인프라 구축 가이드북 (Secure Environment)

이 장에서는 Sentinel Share 서비스의 보안이 적용된 AWS 인프라 환경을 구축한다.

- 1. VPC 설정
- 2. IAM 설정
- 3. ECR 프라이빗 레포지토리 생성
- 4. 보안 그룹 설정
- 5. S3 설정
- 6. GitHub Actions / OIDC 설정
- 7. CloudWatch Log Group 생성
- 8. EC2 인스턴스 생성
- 9. DB 설정
- 10. 백엔드 배포 및 실행
- 11. 프론트엔드 연결 및 접속 테스트
- 12. Secrets Manager 설정
- 13. 자동 배포 및 운영 확인

1. VPC 설정

보안 환경에서는 Public Subnet과 Private Subnet을 분리하여 외부 접근이 필요한 자원과 내부 자원을 구분한다.
secure 환경에서 사용할 명칭은 아래와 같다.

```
VPC: secure-vpc
CIDR: 10.2.0.0/16

Public Subnet: secure-subnet-public
CIDR: 10.2.1.0/24

Private Subnet: secure-subnet-private
CIDR: 10.2.101.0/24

Internet Gateway: secure-igw
NAT Gateway: secure-nat

Public Route Table: secure-rt-public
Private Route Table: secure-rt-private

AZ: ap-northeast-2a
```

1.1 AWS Console 방식

1) VPC 생성

1. AWS Console 접속
2. VPC → Your VPCs
3. Create VPC 클릭
4. 생성 유형에서 VPC only 선택
5. 아래와 같이 입력

설정

Name tag: secure-vpc
IPv4 CIDR: 10.2.0.0/16
Tenancy: Default

생성 후 `secure-vpc` 가 목록에 보이는지 확인한다.

vpc-060814e7e8c3f716f / secure-vpc

작업 ▼

세부 정보 정보

VPC ID
vpc-060814e7e8c3f716f

상태
Available

퍼블릭 액세스 차단
끄기

DNS 호스트 이름
활성화됨

DNS 확인
활성화됨

테넌시
default

DHCP 옵션 세트
dopt-09aac06e203a72001

기본 라우팅 테이블
rtb-063a312af0f57bc50

기본 네트워크 ACL
acl-0fa5d46b2a088c7f3

기본 VPC
아니요

IPv4 CIDR
10.2.0.0/16

IPv6 풀
-

IPv6 CIDR(네트워크 경계 그룹)
-

네트워크 주소 사용 지표
비활성화됨

Route 53 Resolver DNS 방화
벽 규칙 그룹
-

소유자 ID
833453046706

2) Public Subnet 생성

1. VPC → Subnets
2. Create subnet 클릭
3. 아래와 같이 입력

설정

VPC ID: secure-vpc
Subnet name: secure-subnet-public
Availability Zone: ap-northeast-2a
IPv4 subnet CIDR block: 10.2.1.0/24

subnet-0b79ca6f8d6fb8a3b / secure-subnet-public

작업 ▼

세부 정보

서브넷 ID
subnet-0b79ca6f8d6fb8a3b

IPv4 CIDR
10.2.1.0/24

가용 영역
apne2-az1 (ap-northeast-2a)

네트워크 ACL
-

고객 소유 IPv4 주소 자동 할당
아니요

IPv6 CIDR 예약
-

리소스 이름 DNS AAAA 레코드
비활성화됨

서브넷 ARN
arn:aws:ec2:ap-northeast-2:833453046706:subnet/subnet-0b79ca6f8d6fb8a3b

사용 가능한 IPv4 주소
251

네트워크 경계 그룹
ap-northeast-2

기본 서브넷
아니요

고객 소유 IPv4 풀
-

IPv6 전용
아니요

DNS64
비활성화됨

상태
Available

IPv6 CIDR
-

VPC
vpc-060814e7e8c3f716f | secure-vpc

퍼블릭 IPv4 주소 자동 할당
아니요

Outpost ID
-

호스트 이름 유형
IP 이름

소유자
833453046706

퍼블릭 액세스 차단
끄기

IPv6 CIDR 연결 ID
-

라우팅 테이블
rtb-063a312af0f57bc50

IPv6 주소 자동 할당
아니요

IPv4 CIDR 예약
-

리소스 이름 DNS A 레코드
비활성화됨

3) Private Subnet 생성

1. Create subnet 다시 클릭
2. 아래와 같이 입력

설정

VPC ID: secure-vpc

Subnet name: secure-subnet-private

Availability Zone: ap-northeast-2a

IPv4 subnet CIDR block: 10.2.101.0/24

subnet-0463862313b7144e9 / secure-subnet-private

작업 ▼

세부 정보

서브넷 ID
subnet-0463862313b7144e9

IPv4 CIDR
10.2.101.0/24

가용 영역
apne2-az1 (ap-northeast-2a)

네트워크 ACL
-

고객 소유 IPv4 주소 자동 할당
아니요

IPv6 CIDR 예약
-

리소스 이름 DNS AAAA 레코드
비활성화됨

서브넷 ARN
arn:aws:ec2:ap-northeast-2:833453046706:subnet/subnet-0463862313b7144e9

사용 가능한 IPv4 주소
251

네트워크 경계 그룹
ap-northeast-2

기본 서브넷
아니요

고객 소유 IPv4 풀
-

IPv6 전용
아니요

DNS64
비활성화됨

상태
Available

IPv6 CIDR
-

VPC
vpc-060814e7e8c3f716f | secure-vpc

퍼블릭 IPv4 주소 자동 할당
아니요

Outpost ID
-

호스트 이름 유형
IP 이름

소유자
833453046706

퍼블릭 액세스 차단
끄기

IPv6 CIDR 연결 ID
-

라우팅 테이블
rtb-063a312af0f57bc50

IPv6 주소 자동 할당
아니요

IPv4 CIDR 예약
-

리소스 이름 DNS A 레코드
비활성화됨

4) Internet Gateway 생성 및 연결

1. VPC → Internet Gateways
2. Create internet gateway
3. 아래와 같이 입력

설정

Name tag: secure-igw

생성 후 `secure-igw`를 선택하고 **Actions** → **Attach to VPC** 클릭 후 `secure-vpc`에 연결한다.

igw-08522affbc4eacd7a - secure-igw의 인터넷 게이트웨이가 생성되었습니다. 이제 VPC에 연결하여 VPC가 인터넷과 통신하도록 할 수 있습니다.

VPC에 연결

igw-08522affbc4eacd7a / secure-igw

작업

세부 정보

인터넷 게이트웨이 ID
igw-08522affbc4eacd7a

상태
Detached

VPC ID
-

소유자
833453

VPC에 연결
VPC에서 분리
태그 관리
삭제

igw-08522affbc4eacd7a / secure-igw

작업

세부 정보

인터넷 게이트웨이 ID
igw-08522affbc4eacd7a

상태
Attached

VPC ID
vpc-060814e7e8c3f716f / secure-vpc

소유자
833453046706

5) NAT Gateway 생성

1. VPC → NAT Gateways
2. Create NAT gateway
3. 아래와 같이 입력

설정

Name: secure-nat
Subnet: secure-subnet-public
Connectivity type: Public
Elastic IP allocation ID: 새로 할당

NAT Gateway는 반드시 Public Subnet에 생성해야 한다.

nat-05ec6a3da5f23fb3c / secure-nat

작업

세부 정보

NAT 게이트웨이 ID
nat-05ec6a3da5f23fb3c

NAT 게이트웨이 ARN
arn:aws:ec2:ap-northeast-2:833453046706:natgateway/nat-05ec6a3da5f23fb3c

VPC
vpc-060814e7e8c3f716f / secure-vpc

연결 유형
Public

기본 퍼블릭 IPv4 주소
-

서브넷
subnet-0b79ca6f8d6fb8a3b / secure-subnet-public

상태
Pending

기본 프라이빗 IPv4 주소
-

생성됨
2026년 3월 14일 토요일 13시 26분 6초 GMT+9

상태 메시지 정보
-

기본 네트워크 인터페이스 ID
-

삭제됨
-

6) Public Route Table 생성 및 연결

1. VPC → Route Tables
2. Create route table
3. 아래와 같이 입력

설정

Name: secure-rt-public
VPC: secure-vpc

생성 후 `secure-rt-public` 선택

- Routes 탭 → Edit routes
- 라우트 추가

Destination: 0.0.0.0/0
Target: secure-igw

다음으로 Subnet associations 탭 → Edit subnet associations

- `secure-subnet-public` 연결

라우팅 2

대상	대상	상태
0.0.0.0/0	인터넷 게이트웨이	-
igw-08522affbc4eacd7a		
전파됨	라우팅 오리진	
아니요	라우팅 생성	

제거

서브넷 연결 편집

이 라우팅 테이블과 연결된 서브넷을 변경합니다.

이용 가능한 서브넷 (1/2)

이름	서브넷 ID	IPv4 CIDR	IPv6 CIDR	라우팅 테이블
☒ secure-subnet-public	subnet-0b79ca6f8d6fb...	10.2.1.0/24	-	rtb-0ae...
☐ secure-subnet-private	subnet-0463862313b7...	10.2.101.0/24	-	기본 (rtb-0ae...)

선택한 서브넷

subnet-0b79ca6f8d6fb8a3b / secure-subnet-public

취소

연결 저장

rtb-0ae265e2e01e5ea27 / secure-rt-public

작업

세부 정보

라우팅 테이블 ID rtb-0ae265e2e01e5ea27	기본 아니요	명시적 서브넷 연결 subnet-0b79ca6f8d6fb8a3b / secure-subnet-public	옛지 연결 -
VPC vpc-060814e7e8c3f716f secure-vpc	소유자 ID 833453046706		

라우팅 | 서브넷 연결 | 옛지 연결 | 라우팅 전파 | 태그

라우팅 (2)

모두

라우팅 편집

대상	대상	상태	전파됨	라우팅 오리진
0.0.0.0/0	igw-08522affbc4eac...	✓ 활성화	아니요	라우팅 생성
10.2.0.0/16	local	✓ 활성화	아니요	라우팅 테이블 생성

7) Private Route Table 생성 및 연결

1. Create route table
2. 아래와 같이 입력

설정

Name: secure-rt-private
VPC: secure-vpc

생성 후 `secure-rt-private` 선택

- Routes 탭 → Edit routes
- 라우트 추가

Destination: 0.0.0.0/0
Target: secure-nat

다음으로 Subnet associations 탭 → Edit subnet associations

- `secure-subnet-private` 연결

라우팅 2

대상	대상	상태
0.0.0.0/0	NAT 게이트웨이	-
nat-05ec6a3da5f23fb3c		
전파될	라우팅 오리진	
아니요	라우팅 생성	

제거

서브넷 연결 편집

이 라우팅 테이블과 연결된 서브넷을 변경합니다.

이용 가능한 서브넷 (1/2)

서브넷 연결 필터링

이름	서브넷 ID	IPv4 CIDR	IPv6 CIDR	라우팅 테이블
☐ secure-subnet-public	subnet-0b79ca6f8d6fb...	10.2.1.0/24	-	rtb-0ae...
☒ secure-subnet-private	subnet-0463862313b7...	10.2.101.0/24	-	기본 (rtb-0ae...)

선택한 서브넷

subnet-0463862313b7144e9 / secure-subnet-private

취소

연결 저장

세부 정보 정보

라우팅 테이블 ID

rtb-09b3c9cb07fd01a93

기본

아니요

명시적 서브넷 연결

subnet-0463862313b7144e9
/ secure-subnet-private

엣지 연결

-

VPC

vpc-060814e7e8c3f716f |
secure-vpc

소유자 ID

833453046706

라우팅

서브넷 연결

엣지 연결

라우팅 전파

태그

라우팅 (2)

모두 ▼

라우팅 편집

Q 라우팅 필터링

대상	대상	상태	전파될	라우팅 오리진
0.0.0.0/0	nat-05ec6a3da5f23f...	✓ 활성화	아니요	라우팅 생성
10.2.0.0/16	local	✓ 활성화	아니요	라우팅 테이블 생성

8) VPC Peering 생성 (Wazuh 연동)

Wazuh 로그 수집을 위해 main-vpc와 secure-vpc, vul-vpc 간 VPC Peering을 구성한다.

1. VPC → Peering connections
2. Create peering connection 클릭

설정

Name: wazuh-peering-main-secure
Requester VPC: main-vpc
Acceptor VPC: secure-vpc

생성 후 secure-vpc 측에서 피어링 요청을 수락한다.

동일한 방식으로 다음 피어링을 추가로 생성한다.

설정

Name: wazuh-peering-main-vul
Requester VPC: main-vpc
Acceptor VPC: vul-vpc

생성 후 vul-vpc 측에서 피어링 요청을 수락한다.

3. Route Table 경로 추가

각 VPC의 Route Table에 다음 경로를 추가한다.

main-vpc Route Table

Destination: 10.2.0.0/16
Target: wazuh-peering-main-secure

Destination: 10.3.0.0/16
Target: wazuh-peering-main-vul

secure-vpc Route Table

Destination: 10.1.0.0/16
Target: wazuh-peering-main-secure

vul-vpc Route Table

Destination: 10.1.0.0/16
Target: wazuh-peering-main-vul

vul-vpc는 secure 환경 구축에 직접적인 필수 요소는 아니며,
전체 Wazuh 로그 수집 구조 설명을 위한 참고 설정이다.

pcx-03cf9d8fe7218171c / wazuh-peering-main-secure 작업 ▼

세부 정보

요청자 소유자 ID 833453046706	수락자 소유자 ID 833453046706	VPC 피어링 연결 ARN arn:aws:ec2:ap-northeast-2:833453046706:vpc-peering-connection/pcx-03cf9d8fe7218171c
피어링 연결 ID pcx-03cf9d8fe7218171c	요청자 VPC vpc-060814e7e8c3f716f / secure-vpc	수락자 VPC vpc-087c754439a6c7553 / main-vpc
상태 Active	요청자 CIDR 10.2.0.0/16	수락자 CIDR 10.1.0.0/16
만료 시간 -	요청자 리전 서울 (ap-northeast-2)	수락자 리전 서울 (ap-northeast-2)

DNS 라우팅 테이블 태그

DNS 설정 DNS 설정 편집

요청자 VPC(vpc-060814e7e8c3f716f / secure-vpc) 정보

수락자 VPC가 요청자 VPC에서 호스트의 DNS를 프라이빗 IP 주소로 확인하도록 허용
☐ 비활성화됨

수락자 VPC(vpc-087c754439a6c7553 / main-vpc) 정보

요청자 VPC가 수락자 VPC에서 호스트의 DNS를 프라이빗 IP 주소로 확인하도록 허용
☐ 비활성화됨

- main-rt-private에 10.2.0.0/16 -> wazuh-peering-main-secure

라우팅 2

대상: 10.2.0.0/16

전파됨: 아니요

대상: 피어링 연결

상태: 활성

사용: 'pcx-03cf9d8fe7218171c'

pcx-03cf9d8fe7218171c (wazuh-peering-main-secure)

pcx-0c953165b3af30d87 (wazuh-peering-vul2main)

제거

- secure-rt-private에 10.1.0.0/16 -> wazuh-peering-main-secure

라우팅 2

대상: 10.1.0.0/16

전파됨: 아니요

대상: 피어링 연결

상태: 활성

사용: 'pcx-03cf9d8fe7218171c'

pcx-03cf9d8fe7218171c (wazuh-peering-main-secure)

제거

1.2 CLI 방식

먼저 공통 변수 예시를 지정한다.


```
AWS_REGION=ap-northeast-2
AWS_AZ=ap-northeast-2a
```

1) VPC 생성

```
VPC_ID=$(aws ec2 create-vpc \
  --region$AWS_REGION \
  --cidr-block 10.2.0.0/16 \
  --tag-specifications"ResourceType=vpc,Tags=[{Key=Name,Value=secure-vpc}]" \
  --query'Vpc.VpcId' \
  --output text)
```

2) Public Subnet 생성

```
PUBLIC_SUBNET_ID=$(aws ec2 create-subnet \
  --region$AWS_REGION \
  --vpc-id$VPC_ID \
  --cidr-block 10.2.1.0/24 \
  --availability-zone$AWS_AZ \
  --tag-specifications"ResourceType=subnet,Tags=[{Key=Name,Value=secure-subnet-public}]" \
  --query'Subnet.SubnetId' \
  --output text)
```

퍼블릭 서브넷에서 퍼블릭 IP 자동 할당을 활성화한다.

```
aws ec2 modify-subnet-attribute \
  --region$AWS_REGION \
  --subnet-id$PUBLIC_SUBNET_ID \
  --map-public-ip-on-launch
```

3) Private Subnet 생성

```
PRIVATE_SUBNET_ID=$(aws ec2 create-subnet \
  --region$AWS_REGION \
  --vpc-id$VPC_ID \
  --cidr-block 10.2.101.0/24 \
  --availability-zone$AWS_AZ \
  --tag-specifications"ResourceType=subnet,Tags=[{Key=Name,Value=secure-subnet-private}]" \
  --query'Subnet.SubnetId' \
  --output text)
```

4) Internet Gateway 생성 및 연결

```
IGW_ID=$(aws ec2 create-internet-gateway \
  --region$AWS_REGION \
  --tag-specifications"ResourceType=internet-gateway,Tags=[{Key=Name,Value=secure-igw}]" \
  --query'InternetGateway.InternetGatewayId' \
  --output text)
```

```
aws ec2 attach-internet-gateway \
--region$AWS_REGION \
--internet-gateway-id$IGW_ID \
--vpc-id$VPC_ID
```

5) Elastic IP 할당 및 NAT Gateway 생성

```
EIP_ALLOC_ID=$(aws ec2 allocate-address \
--region$AWS_REGION \
--domain vpc \
--query'AllocationId' \
--output text)

NAT_GW_ID=$(aws ec2 create-nat-gateway \
--region$AWS_REGION \
--subnet-id$PUBLIC_SUBNET_ID \
--allocation-id$EIP_ALLOC_ID \
--tag-specifications"ResourceType=natgateway,Tags=[{Key=Name,Value=secure-nat}]" \
--query'NatGateway.NatGatewayId' \
--output text)
```

NAT Gateway가 **available** 상태가 될 때까지 기다린다.

```
aws ec2 wait nat-gateway-available \
--region$AWS_REGION \
--nat-gateway-ids$NAT_GW_ID
```

6) Public Route Table 생성 및 연결

```
PUBLIC_RT_ID=$(aws ec2 create-route-table \
--region$AWS_REGION \
--vpc-id$VPC_ID \
--tag-specifications"ResourceType=route-table,Tags=[{Key=Name,Value=secure-rt-public}]" \
--query'RouteTable.RouteTableId' \
--output text)

aws ec2 create-route \
--region$AWS_REGION \
--route-table-id$PUBLIC_RT_ID \
--destination-cidr-block0.0.0.0/0 \
--gateway-id$IGW_ID

aws ec2 associate-route-table \
--region$AWS_REGION \
--route-table-id$PUBLIC_RT_ID \
--subnet-id$PUBLIC_SUBNET_ID
```

7) Private Route Table 생성 및 연결

```
PRIVATE_RT_ID=$(aws ec2 create-route-table \
--region$AWS_REGION \
--vpc-id$VPC_ID \
--tag-specifications"ResourceType=route-table,Tags=[{Key=Name,Value=secure-rt-privat
```

```
e}}" \
  --query 'RouteTable.RouteTableId' \
  --output text)

aws ec2 create-route \
  --region $AWS_REGION \
  --route-table-id $PRIVATE_RT_ID \
  --destination-cidr-block 0.0.0.0/0 \
  --nat-gateway-id $NAT_GW_ID

aws ec2 associate-route-table \
  --region $AWS_REGION \
  --route-table-id $PRIVATE_RT_ID \
  --subnet-id $PRIVATE_SUBNET_ID
```

8) VPC Peering 생성

```
MAIN_VPC_ID=main-vpc의 VPC ID
VUL_VPC_ID=vul-vpc의 VPC ID
MAIN_PRIVATE_RT_ID=main-vpc의 Route Table ID
VUL_PRIVATE_RT_ID=vul-vpc의 Route Table ID
```

main ↔ secure peering

```
MAIN_SECURE_PEERING_ID=$(aws ec2 create-vpc-peering-connection \
  --region $AWS_REGION \
  --vpc-id $MAIN_VPC_ID \
  --peer-vpc-id $VPC_ID \
  --tag-specifications "ResourceType=vpc-peering-connection,Tags=[{Key=Name,Value=wazu
h-peering-main-secure}]" \
  --query 'VpcPeeringConnection.VpcPeeringConnectionId' \
  --output text)
```

수락

```
aws ec2 accept-vpc-peering-connection \
  --region $AWS_REGION \
  --vpc-peering-connection-id $MAIN_SECURE_PEERING_ID
```

vul-vpc 역시 동일한 방식으로 main-vpc와 Peering을 구성한다.

이는 secure 환경 구축에 직접적인 필수 요소는 아니며,

전체 Wazuh 로그 수집 구조 설명을 위한 참고 설정이다.

main ↔ vul peering

```
MAIN_VUL_PEERING_ID=$(aws ec2 create-vpc-peering-connection \
  --region $AWS_REGION \
  --vpc-id $MAIN_VPC_ID \
  --peer-vpc-id $VUL_VPC_ID \
  --tag-specifications "ResourceType=vpc-peering-connection,Tags=[{Key=Name,Value=wazu
h-peering-main-vul}]" \
  --query 'VpcPeeringConnection.VpcPeeringConnectionId' \
  --output text)
```

수락

```
aws ec2 accept-vpc-peering-connection \
--region $AWS_REGION \
--vpc-peering-connection-id $MAIN_VUL_PEERING_ID
```

NOTE

보안 양호 환경에서 VPC를 이렇게 구성하는 이유는 **외부 통신이 필요한 자원과 내부 보호가 필요한 자원을 분리하기 위해서**이다.

- secure-subnet-public은 NAT Gateway, Bastion EC2처럼 외부 통신이 필요한 자원에 사용한다.
- secure-subnet-private은 ECS, RDS 같은 핵심 자원을 배치하는 내부 전용 서브넷으로 사용한다.
- secure-rt-public은 0.0.0.0/0 -> secure-igw로 퍼블릭 서브넷을 인터넷과 연결한다.
- secure-rt-private은 0.0.0.0/0 -> secure-nat로 프라이빗 서브넷이 외부로만 통신할 수 있게 한다.

추가로 Wazuh 연동을 위해 main-vpc와 secure-vpc 간 VPC Peering을 구성할 수 있다.

이 경우 피어링 연결만 생성하는 것으로 끝나지 않고, main-vpc Route Table에는 10.2.0.0/16, secure-vpc Route Table에는 10.1.0.0/16 경로를 함께 추가해야 실제 통신이 가능하다.

참고로 main-vpc ↔ vul-vpc 피어링은 secure 환경 구축의 직접적인 필수 요소는 아니며, 전체 Wazuh 로그 수집 구조 설명을 위한 참고 설정이다.

취약한 예시는 다음과 같다.

- DB를 Public Subnet에 배치
- Private Subnet을 Internet Gateway에 직접 연결
- NAT 없이 Private Subnet만 구성
- VPC Peering만 만들고 Route Table 경로를 추가하지 않음



체크리스트

- **secure-vpc** 가 생성되었는가
- **secure-subnet-public** , **secure-subnet-private** 가 모두 생성되었는가
- 두 서브넷의 AZ가 모두 **ap-northeast-2a** 인가
- **secure-igw** 가 **secure-vpc** 에 연결되었는가
- **secure-nat** 가 **secure-subnet-public** 에 생성되었는가
- **secure-rt-public** 의 기본 라우트가 IGW를 가리키는가
- **secure-rt-private** 의 기본 라우트가 NAT Gateway를 가리키는가
- Public Route Table은 Public Subnet에, Private Route Table은 Private Subnet에 연결되었는가
- **main-vpc ↔ secure-vpc** VPC Peering이 생성되고 수락되었는가
- **main-vpc ↔ vul-vpc** VPC Peering이 생성되고 수락되었는가
- **main-vpc** Route Table에 **10.2.0.0/16** , **10.3.0.0/16** 경로가 추가되었는가
- **secure-vpc** Route Table에 **10.1.0.0/16** 경로가 추가되었는가
- **vul-vpc** Route Table에 **10.1.0.0/16** 경로가 추가되었는가

2. IAM 설정

secure 환경에서 EC2 인스턴스가 S3, Secrets Manager, SSM 등에 접근할 수 있도록 IAM 리소스를 생성한다.

사용 이름

- secure-ec2-role

2.1 AWS Console 방식

1) EC2 IAM Role 생성

1. AWS Console 접속
2. IAM → Roles
3. Create role
4. 신뢰할 수 있는 엔티티 유형에서 AWS service 선택
5. 사용 사례에서 EC2 선택
6. 다음 정책 연결

AmazonEC2ContainerRegistryReadOnly

AmazonSSMManagedInstanceCore
CloudWatchAgentServerPolicy

7. 역할 이름 입력 `secure-ec2-role`
8. 생성 완료 후, 역할 상세 화면에서 권한 추가 → 인라인 정책 생성 → JSON 으로 이동한다.
9. 아래 정책을 붙여넣고 정책 이름을 secure-ec2-inline으로 저장한다.

secure-ec2-inline

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "AllowAppBucketObjects",
      "Effect": "Allow",
      "Action": [
        "s3:GetObject",
        "s3:PutObject",
        "s3:DeleteObject"
      ],
      "Resource": "arn:aws:s3:::secure-files-private/uploads/*"
    },
    {
      "Sid": "AllowAppBucketList",
      "Effect": "Allow",
      "Action": [
        "s3:ListBucket"
      ],
      "Resource": "arn:aws:s3:::secure-files-private"
    },
    {
      "Sid": "AllowSpecificSecretsRead",
      "Effect": "Allow",
      "Action": [
        "secretsmanager:GetSecretValue"
      ]
    }
  ]
}
```

```

    ],
    "Resource": [
      "arn:aws:secretsmanager:ap-northeast-2:YOUR_ACCOUNT_ID:secret:secure-db-password-*",
      "arn:aws:secretsmanager:ap-northeast-2:YOUR_ACCOUNT_ID:secret:secure-jwt-secret-*"
    ]
  }
}

```

secure-ec2-role 정보 삭제

secure-ec2-role

요약

생성 날짜
March 16, 2026, 09:32 (UTC+09:00)

마지막 활동
🟢 17분 전

ARN
arn:aws:iam::833453046706:role/secure-ec2-role

최대 세션 지속 시간
1시간

인스턴스 프로파일 ARN
arn:aws:iam::833453046706:instance-profile/secure-ec2-role

편집

[권한](#) | [신뢰 관계](#) | [태그](#) | [마지막 액세스](#) | [세션 취소](#)

권한 정책 (4) 정보 시뮬레이션 삭제 권한 추가

최대 10개의 관리형 정책을 연결할 수 있습니다.

필터링 기준 유형: 모든 유형

☐	정책 이름	유형	연결된 엔터티
☐	AmazonEC2ContainerRegistryReadOnly	AWS 관리형	1
☐	AmazonSSMManagedInstanceCore	AWS 관리형	1
☐	CloudWatchAgentServerPolicy	AWS 관리형	1
☐	secure-ec2-inline	고객 인라인	0

2.2 CLI 방식

먼저 신뢰 정책 파일을 만든다.

trust-policy-ec2.json

```

{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Principal": { "Service": "ec2.amazonaws.com" },
      "Action": "sts:AssumeRole"
    }
  ]
}

```

secure-ec2-inline-policy.json

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "AllowAppBucketObjects",
      "Effect": "Allow",
      "Action": [
        "s3:GetObject",
        "s3:PutObject",
        "s3:DeleteObject"
      ],
      "Resource": "arn:aws:s3:::secure-files-private/uploads/*"
    },
    {
      "Sid": "AllowAppBucketList",
      "Effect": "Allow",
      "Action": [
        "s3:ListBucket"
      ],
      "Resource": "arn:aws:s3:::secure-files-private"
    },
    {
      "Sid": "AllowSpecificSecretsRead",
      "Effect": "Allow",
      "Action": [
        "secretsmanager:GetSecretValue"
      ],
      "Resource": [
        "arn:aws:secretsmanager:ap-northeast-2:YOUR_ACCOUNT_ID:secret:secure-db-password-*",
        "arn:aws:secretsmanager:ap-northeast-2:YOUR_ACCOUNT_ID:secret:secure-jwt-secret-*"
      ]
    }
  ]
}
```

1) EC2 Role 생성

```
aws iam create-role \
  --role-name secure-ec2-role \
  --assume-role-policy-document file://trust-policy-ec2.json

aws iam attach-role-policy \
  --role-name secure-ec2-role \
  --policy-arn arn:aws:iam::aws:policy/AmazonSSMManagedInstanceCore

aws iam attach-role-policy \
  --role-name secure-ec2-role \
  --policy-arn arn:aws:iam::aws:policy/CloudWatchAgentServerPolicy

aws iam put-role-policy \
  --role-name secure-ec2-role \
```

```
--policy-name secure-ec2-inline \  
--policy-document file://secure-ec2-inline-policy.json
```

NOTE

- secure-ec2-role은 EC2가 S3, Secrets Manager, SSM에 접근할 때 사용한다.
- 보안 양호 환경에서는 전체 권한보다 버킷, 경로, 시크릿 단위 최소 권한이 맞다.
- EC2에 직접 SSH만 의존하기보다 SSM 접속도 함께 준비하는 것이 좋다.

체크리스트

- secure-ec2-role이 생성되었는가
- AmazonSSMManagedInstanceCore가 연결되었는가
- CloudWatchAgentServerPolicy가 연결되었는가
- secure-ec2-role에 인라인 정책이 추가되었는가
- 시크릿 및 S3 접근 범위가 필요한 리소스로 제한되었는가

3. ECR 프라이빗 레포지토리 생성

컨테이너 이미지를 저장하기 위해 ECR 레포지토리를 생성한다.

사용 이름

```
s3cure-api
```

3.1 AWS Console 방식

1. AWS Console 접속
2. Amazon ECR → Repositories
3. Create repository
4. 아래와 같이 입력

설정

```
Visibility settings: Private  
Repository name: s3cure-api  
Image tag mutability: Mutable  
Scan on push: On  
Encryption: AES-256
```

생성 후 Repository URI를 확인해 둔다.

이 값은 이후 ECS Task Definition에서 사용한다.

예시

```
123456789012.dkr.ecr.ap-northeast-2.amazonaws.com/s3cure-api
```


s3cure-api

[요약](#)[이미지](#)[Lifecycle policy](#)[Permissions](#)[리포지토리 태그](#)

리포지토리 세부 정보

[리로드](#) [편집](#)

리포지토리 이름
s3cure-api

생성 시간
2026년 3월 15일, 19:12:36 (UTC+09)

리포지토리 ARN
arn:aws:ecr:ap-northeast-2:833453046706:repository/s3cure-api

리포지토리 URI
833453046706.dkr.ecr.ap-northeast-2.amazonaws.com/s3cure-api

태그 변경 가능성
변경 가능

태그 변경 가능성 제외
-

암호화 유형
AES-256

스캔 빈도
푸시할 때 스캔

3.2 CLI 방식

```
ECR_URI=$(aws ecr create-repository \
  --repository-name s3cure-api \
  --image-scanning-configuration scanOnPush=true \
  --region ap-northeast-2 \
  --query 'repository.repositoryUri' \
  --output text)
```

로그인

```
aws ecr get-login-password \
  --region ap-northeast-2 \
  | docker login \
  --username AWS \
  --password-stdin ${ECR_URI}/*}
```

이미지 push 예시

```
docker tag sentinel-api:latest ${ECR_URI}:latest
docker push ${ECR_URI}:latest
```

NOTE

ECR은 ECS에서 사용할 이미지를 저장하는 공간이다.

- **Private**로 설정하여 외부 공개를 막는다.
- **Scan on push**를 활성화하면 이미지를 푸시할 때 취약점 점검이 수행된다.
- 이후 ECS는 이 저장소의 이미지를 pull하여 컨테이너를 실행한다.

취약한 예시는 다음과 같다.

- 불필요하게 Public Repository 사용
- 이미지 스캔 비활성화
- 검증되지 않은 이미지를 바로 배포

체크리스트

- **s3cure-api** 레포지토리가 생성되었는가
- Visibility가 **Private**인가

- Push 시 스캔이 활성화되었는가
- Repository URI를 확인했는가
- 테스트 이미지가 정상적으로 push 되었는가

4. 보안 그룹 설정

보안 그룹은 네트워크 접근을 제어한다.

여기서는 아래 3개를 생성한다.

- secure-ec2-sg
- secure-bastion-sg

4.1 AWS Console 방식

1) EC2 보안 그룹 생성

1. AWS Console 접속
2. EC2 → Security Groups
3. Create security group
4. 아래와 같이 입력

설정

```
Security group name: secure-ec2-sg
Description: security group for secure ec2
VPC: secure-vpc
```

인바운드 규칙 추가

```
Type: SSH
Port range: 22
Source: 내 IP/32
```

```
Type: Custom TCP
Port range: 3000
Source: 내 IP/32
```

sg-0b816946a9763ef5f - secure-ec2-sg

[작업 ▼](#)

세부 정보

보안 그룹 이름

[secure-ec2-sg](#)

보안 그룹 ID

[sg-0b816946a9763ef5f](#)

설명

[security group for secure pub](#)
lic ec2

VPC ID

[vpc-060814e7e8c3f716f](#)

소유자

[833453046706](#)

인바운드 규칙 수

2 권한 항목

아웃바운드 규칙 수

1 권한 항목

[인바운드 규칙](#)[아웃바운드 규칙](#)[공유](#)[VPC 연결](#)[관련 리소스 - 신규](#)[태그](#)

인바운드 규칙 (2)

[태그 관리](#)[인바운드 규칙 편집](#)

Q 검색

< 1 >



☐	Name ▼	보안 그룹 규칙 ID ▼	IP 버전 ▼	유형 ▼	프로토콜 ▼	포트 범위 ▼	소스
☐	-	sg-0c46f35daa5fb78a2	IPv4	사용자 지정 TCP	TCP	3000	112.218.95.58/32
☐	-	sg-0b568f5ee3c16433f	IPv4	SSH	TCP	22	112.218.95.58/32

2) Bastion 보안 그룹 생성

1. Create security group

2. 아래와 같이 입력

설정

Security group name: secure-bastion-sg

Description: security group for secure bastion

VPC: secure-vpc

인바운드 규칙 추가

Type: SSH

Port range: 22

Source: 내 IP/32

sg-091961404518c6921 - secure-bastion-sg

[작업 ▼](#)

세부 정보

보안 그룹 이름

[secure-bastion-sg](#)

보안 그룹 ID

[sg-091961404518c6921](#)

설명

[security group for secure bas](#)
tion

VPC ID

[vpc-060814e7e8c3f716f](#)

소유자

[833453046706](#)

인바운드 규칙 수

1 권한 항목

아웃바운드 규칙 수

1 권한 항목

[인바운드 규칙](#)[아웃바운드 규칙](#)[공유](#)[VPC 연결](#)[관련 리소스 - 신규](#)[태그](#)

인바운드 규칙 (1)

[태그 관리](#)[인바운드 규칙 편집](#)

Q 검색

< 1 >



☐	Name ▼	보안 그룹 규칙 ID ▼	IP 버전 ▼	유형 ▼	프로토콜 ▼	포트 범위 ▼	소스
☐	-	sg-038edc44649e6f1bf	IPv4	SSH	TCP	22	211.217.209.136/32

4.2 CLI 방식

1) Container 보안 그룹

```
CONTAINER_SG_ID=$(aws ec2 create-security-group \
  --group-name secure-container-sg \
  --description "security group for secure container" \
  --vpc-id $VPC_ID \
  --region $AWS_REGION \
  --query 'GroupId' \
  --output text)
```

인바운드 규칙 추가

```
aws ec2 authorize-security-group-ingress \
  --group-id $CONTAINER_SG_ID \
  --protocol tcp \
  --port 3000 \
  --cidr 10.1.0.0/16 \
  --region $AWS_REGION
```

2) Bastion 보안 그룹

```
MY_IP=YOUR_PUBLIC_IP/32
```

```
BASTION_SG_ID=$(aws ec2 create-security-group \
  --group-name secure-bastion-sg \
  --description "security group for secure bastion" \
  --vpc-id $VPC_ID \
  --region $AWS_REGION \
  --query 'GroupId' \
  --output text)
```

인바운드 규칙 추가

```
aws ec2 authorize-security-group-ingress \
  --group-id $BASTION_SG_ID \
  --protocol tcp \
  --port 22 \
  --cidr $MY_IP \
  --region $AWS_REGION
```

NOTE

- secure-ec2는 실습상 public subnet에 두되, SSH와 애플리케이션 포트 모두 내 IP/32로 제한한다.
- 0.0.0.0/0 공개는 보안 양호 기준에 맞지 않는다.
- 여러 사용자가 접속해야 하면 이후 ALB를 앞단에 두고 EC2 SG를 더 좁히는 것이 적절하다.

체크리스트

- secure-ec2-sg가 생성되었는가
- SSH 22 포트가 내 IP/32로 제한되었는가
- 3000 포트가 내 IP/32로 제한되었는가
- secure-ec2에 secure-ec2-sg가 연결되었는가

5. S3 설정

로그 저장용 버킷을 생성한다.

사용 이름

secure-log

5.1 AWS Console 방식

1. AWS Console 접속
2. Amazon S3 → Create bucket
3. 아래와 같이 입력

설정

```
Bucket name: secure-log
Region: ap-northeast-2
Block Public Access: On
Bucket Versioning: 선택 사항
Default encryption: SSE-S3 또는 SSE-KMS
```

이 버킷은 실습 중 생성되는 로그나 메타데이터를 저장하는 용도로 사용할 수 있다.

예를 들어 아래와 같은 용도로 활용할 수 있다.

- 애플리케이션 업로드 로그
- 보안 점검 결과 파일
- 공격 시나리오 실행 결과
- CloudTrail / ALB / S3 Access Log 연계 저장

secure-log-833453046706-ap-northeast-2-an 정보

객체 | 메타데이터 | 속성 | **권한** | 지표 | 관리 | 액세스 지점

권한 개요

액세스 찾기

액세스 조사 결과는 IAM 외부 액세스 분석기에서 제공합니다. [IAM 분석기 조사 결과가 작동하는 방식](#)에 대해 자세히 알아보세요.
[ap-northeast-2에 대한 분석기 보기](#)

퍼블릭 액세스 차단(버킷 설정)

편집

퍼블릭 액세스는 ACL(엑세스 제어 목록), 버킷 정책, 액세스 지점 정책 또는 모두를 통해 버킷 및 객체에 부여됩니다. 모든 S3 버킷 및 객체에 대한 퍼블릭 액세스가 차단되었는지 확인하려면 [모든 퍼블릭 액세스 차단]을 활성화합니다. 이 설정은 이 버킷 및 해당 액세스 지점에만 적용됩니다. AWS에서는 [모든 퍼블릭 액세스 차단]을 활성화하도록 권장하지만, 이 설정을 적용하기 전에 퍼블릭 액세스가 없어도 애플리케이션이 올바르게 작동하는지 확인합니다. 버킷 또는 내부 객체에 어느 정도 수준의 퍼블릭 액세스가 필요한 경우 특정 스토리지 사용 사례에 맞게 아래 개별 설정을 사용자 지정할 수 있습니다. [자세히 알아보기](#)

모든 퍼블릭 액세스 차단

🟢 활성화

▶ 이 버킷의 개별 퍼블릭 액세스 차단 설정

5.2 CLI 방식

```
aws s3api create-bucket \
  --bucket secure-log \
  --region $AWS_REGION \
  --create-bucket-configuration LocationConstraint=$AWS_REGION
```

퍼블릭 액세스 차단

```
aws s3api put-public-access-block \
  --bucket secure-log \
  --public-access-block-configuration \
    BlockPublicAcls=true,IgnorePublicAcls=true,BlockPublicPolicy=true,RestrictPublicBuckets=true
```

기본 암호화 적용

```
aws s3api put-bucket-encryption \
  --bucket secure-log \
  --server-side-encryption-configuration \
    '{"Rules":[{"ApplyServerSideEncryptionByDefault":{"SSEAlgorithm":"AES256"}}]}'
```

NOTE

보안 양호 환경의 S3 버킷은 기본적으로 외부 공개를 막아야 한다.

- secure-log는 로그나 결과 저장용 버킷으로 사용할 수 있다.
- 퍼블릭 액세스를 차단하면 실수로 로그 파일이 외부에 노출되는 위험을 줄일 수 있다.
- 암호화를 적용하면 저장 데이터 보호에 도움이 된다.

체크리스트

- secure-log 버킷이 생성되었는가
- 퍼블릭 액세스 차단이 활성화되었는가
- 기본 암호화가 적용되었는가
- 버킷 목적을 로그/결과 저장용으로 정했는가

6. GitHub Actions / OIDC 설정

ECR push 자동화를 위해 GitHub Actions와 AWS OIDC를 설정한다.

6.1 AWS Console 방식

1) OIDC Provider 생성

1. AWS Console 접속
2. IAM → Identity providers
3. Add provider
4. 아래와 같이 입력

설정

```
Provider type: OpenID Connect
Provider URL: https://token.actions.githubusercontent.com
Audience: sts.amazonaws.com
```

token.actions.githubusercontent.com
정보
역할 할당
삭제

요약

공급자 token.actions.githubusercontent.com	공급자 유형 OpenID Connect	생성 시간 March 13, 2026, 14:23 (UTC+09:00)	ARN arn:aws:iam::833453046706:oidc-provider/token.actions.githubusercontent.com
---	---------------------------------	---	---

2) GitHub Actions용 IAM Role 생성

1. IAM → Roles → Create role
2. 신뢰할 수 있는 엔터티 유형에서 Web identity 선택
3. 아래와 같이 입력

설정

```
Identity provider: token.actions.githubusercontent.com
Audience: sts.amazonaws.com
GitHub organization: CloudShield-Lab
GitHub repository: CloudShield_Lab
```

1. 다음으로 이동
2. 역할 이름 입력 CloudShield-Role
3. Create role 클릭

이 역할은 GitHub Actions가 AssumeRole 하여 ECR push 및 필요한 배포 작업을 수행할 때 사용됩니다.

CloudShield-Role
정보
삭제

요약

생성 날짜
March 13, 2026, 15:21 (UTC+09:00)

마지막 활동
2일 전

ARN
arn:aws:iam::833453046706:role/CloudShield-Role

최대 세션 지속 시간
1시간

편집

3) Trust relationship 확인 및 저장소 제한 설정

역할 생성 후 CloudShield-Role → Trust relationships에서 신뢰 정책을 확인한다.

신뢰 정책은 아래와 같은 형태로 저장소를 제한해야 한다.

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Principal": {
        "Federated": "arn:aws:iam::833453046706:oidc-provider/token.actions.githubusercontent.com"
      },
      "Action": "sts:AssumeRoleWithWebIdentity",
      "Condition": {
        "StringEquals": {
          "token.actions.githubusercontent.com:aud": "sts.amazonaws.com"
        }
      }
    }
  ]
}
```

```

    },
    "StringLike": {
      "token.actions.githubusercontent.com:sub": "repo:CloudShield-Lab/CloudShield
_Lab:*"
    }
  }
}
]
}

```

설명:

- aud는 반드시 sts.amazonaws.com
- sub는 반드시 특정 GitHub 저장소 기준으로 제한
- 필요하면 브랜치까지 더 좁게 제한 가능

4) CloudShield-Policy 생성 및 연결

1. IAM → Policies
2. Create policy
3. JSON 탭 선택
4. 아래 정책 내용을 붙여넣는다.
5. 정책 이름 입력 `CloudShield-Policy`
6. 정책 생성 후, CloudShield-Role에 연결한다.

정책 예시

```

{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "ECRPushPull",
      "Effect": "Allow",
      "Action": [
        "ecr:GetAuthorizationToken",
        "ecr:BatchCheckLayerAvailability",
        "ecr:CompleteLayerUpload",
        "ecr:InitiateLayerUpload",
        "ecr:UploadLayerPart",
        "ecr:PutImage",
        "ecr:BatchGetImage"
      ],
      "Resource": "*"
    },
    {
      "Sid": "ECSDeploy",
      "Effect": "Allow",
      "Action": [
        "ecs:DescribeTaskDefinition",
        "ecs:RegisterTaskDefinition",
        "ecs:DeregisterTaskDefinition",
        "ecs:UpdateService",
        "ecs:DescribeServices",
        "ecs:DescribeClusters",

```



```

        "ecs:DescribeTasks",
        "ecs:ListTasks"
    ],
    "Resource": "*"
},
{
    "Sid": "PassOnlyECSTaskRoles",
    "Effect": "Allow",
    "Action": "iam:PassRole",
    "Resource": [
        "arn:aws:iam::833453046706:role/*"
    ],
    "Condition": {
        "StringEquals": {
            "iam:PassedToService": "ecs-tasks.amazonaws.com"
        }
    }
},
{
    "Sid": "TerraformInfraCore",
    "Effect": "Allow",
    "Action": [
        "ec2:*",
        "elasticloadbalancing:*",
        "ecs:*",
        "logs:*",
        "iam:GetRole",
        "iam:CreateRole",
        "iam>DeleteRole",
        "iam:AttachRolePolicy",
        "iam:DetachRolePolicy",
        "iam:PutRolePolicy",
        "iam>DeleteRolePolicy",
        "iam:TagRole",
        "iam:UntagRole",
        "ecr:DescribeRepositories",
        "ecr:CreateRepository",
        "ecr>DeleteRepository",
        "ecr:TagResource",
        "ecr:UntagResource",
        "ecr:SetRepositoryPolicy",
        "ecr>DeleteRepositoryPolicy",
        "wafv2:*"
    ],
    "Resource": "*"
},
{
    "Sid": "TerraformStateS3",
    "Effect": "Allow",
    "Action": [
        "s3:ListBucket",
        "s3:GetObject",
        "s3:PutObject",
        "s3:DeleteObject"
    ],

```

```

    "Resource": [
      "arn:aws:s3:::<TF_STATE_BUCKET>",
      "arn:aws:s3:::<TF_STATE_BUCKET>/*"
    ]
  },
  {
    "Sid": "TerraformStateLockDynamoDB",
    "Effect": "Allow",
    "Action": [
      "dynamodb:GetItem",
      "dynamodb:PutItem",
      "dynamodb>DeleteItem",
      "dynamodb:UpdateItem",
      "dynamodb:DescribeTable"
    ],
    "Resource": "arn:aws:dynamodb:ap-northeast-2:833453046706:table/<TF_LOCK_TABLE>"
  },
  {
    "Sid": "ECRReadOnly",
    "Effect": "Allow",
    "Action": [
      "ecr:GetAuthorizationToken",
      "ecr:DescribeRepositories",
      "ecr:DescribeImages",
      "ecr:ListImages",
      "ecr:BatchGetImage",
      "ecr:GetDownloadUrlForLayer"
    ],
    "Resource": "*"
  },
  {
    "Sid": "SSMCoreAccess",
    "Effect": "Allow",
    "Action": [
      "ssm:DescribeInstanceInformation",
      "ssm:ListCommandInvocations",
      "ssm:ListCommands",
      "ssm:SendCommand",
      "ssm:GetCommandInvocation",
      "ssm:StartSession",
      "ssm:TerminateSession",
      "ssm:ResumeSession",
      "ssm:GetConnectionStatus"
    ],
    "Resource": "*"
  }
]
}

```

CloudShield-Policy 정보

[편집](#)[삭제](#)

CloudShield-Policy

정책 세부 정보

유형
고객 관리형

생성 시간
March 13, 2026, 15:21
(UTC+09:00)

편집 시간
March 13, 2026, 17:47
(UTC+09:00)

ARN
 arn:aws:iam::833453046706:policy/CloudShield-Policy

[권한](#)[연결된 엔터티](#)[태그](#)[정책 버전 \(5\)](#)[마지막 액세스](#)

권한 정책으로 연결됨 (1)

[연결](#)[분리](#)

엔터티에 권한을 부여하려면 해당 엔터티에 권한 정책을 연결합니다.

필터링 기준 엔터티 유형

모든 유형

< 1 > ☐ 엔터티 이름

▲ | 엔터티 유형

▼ |

☐ CloudShield-Role

역할

5) GitHub Secrets 등록

GitHub 저장소에서

Settings → **Secrets and variables** → **Actions**

아래 값을 등록한다.

`AWS_ACCOUNT_ID``AWS_REGION``AWS_ROLE_ARN`

예시 값

`AWS_ACCOUNT_ID = 833453046706``AWS_REGION = ap-northeast-2``AWS_ROLE_ARN = arn:aws:iam::833453046706:role/CloudShield-Role`

Repository secrets		New repository secret	
Name ↕		Last updated	
 <code>AWS_ACCOUNT_ID</code>		1 minute ago	 
 <code>AWS_REGION</code>		1 minute ago	 
 <code>AWS_ROLE_ARN</code>		2 days ago	 

6.2 CLI 방식

OIDC Provider는 콘솔에서 설정하는 편이 실습상 더 편하다.

IAM Role과 정책은 CLI로도 생성할 수 있다.

1) 신뢰 정책 파일 생성

파일명 `github-oidc-trust.json`

```
{
  "Version": "2012-10-17",
  "Statement": [
```

```

    {
      "Effect": "Allow",
      "Principal": {
        "Federated": "arn:aws:iam::833453046706:oidc-provider/token.actions.githubusercontent.com"
      },
      "Action": "sts:AssumeRoleWithWebIdentity",
      "Condition": {
        "StringEquals": {
          "token.actions.githubusercontent.com:aud": "sts.amazonaws.com"
        },
        "StringLike": {
          "token.actions.githubusercontent.com:sub": "repo:CloudShield-Lab/CloudShield
_Lab:*"
        }
      }
    }
  ]
}

```

2) 정책 파일 생성

파일명 cloudshield-policy.json

```

{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "ECRPushPull",
      "Effect": "Allow",
      "Action": [
        "ecr:GetAuthorizationToken",
        "ecr:BatchCheckLayerAvailability",
        "ecr:CompleteLayerUpload",
        "ecr:InitiateLayerUpload",
        "ecr:UploadLayerPart",
        "ecr:PutImage",
        "ecr:BatchGetImage"
      ],
      "Resource": "*"
    },
    {
      "Sid": "ECSDeploy",
      "Effect": "Allow",
      "Action": [
        "ecs:DescribeTaskDefinition",
        "ecs:RegisterTaskDefinition",
        "ecs:DeregisterTaskDefinition",
        "ecs:UpdateService",
        "ecs:DescribeServices",
        "ecs:DescribeClusters",
        "ecs:DescribeTasks",
        "ecs:ListTasks"
      ],
      "Resource": "*"
    }
  ]
}

```

```

},
{
  "Sid": "PassOnlyECSTaskRoles",
  "Effect": "Allow",
  "Action": "iam:PassRole",
  "Resource": [
    "arn:aws:iam::833453046706:role/*"
  ],
  "Condition": {
    "StringEquals": {
      "iam:PassedToService": "ecs-tasks.amazonaws.com"
    }
  }
},
{
  "Sid": "TerraformInfraCore",
  "Effect": "Allow",
  "Action": [
    "ec2:*",
    "elasticloadbalancing:*",
    "ecs:*",
    "logs:*",
    "iam:GetRole",
    "iam:CreateRole",
    "iam>DeleteRole",
    "iam:AttachRolePolicy",
    "iam:DetachRolePolicy",
    "iam:PutRolePolicy",
    "iam>DeleteRolePolicy",
    "iam:TagRole",
    "iam:UntagRole",
    "ecr:DescribeRepositories",
    "ecr:CreateRepository",
    "ecr>DeleteRepository",
    "ecr:TagResource",
    "ecr:UntagResource",
    "ecr:SetRepositoryPolicy",
    "ecr>DeleteRepositoryPolicy",
    "wafv2:*"
  ],
  "Resource": "*"
},
{
  "Sid": "TerraformStateS3",
  "Effect": "Allow",
  "Action": [
    "s3:ListBucket",
    "s3:GetObject",
    "s3:PutObject",
    "s3:DeleteObject"
  ],
  "Resource": [
    "arn:aws:s3:::<TF_STATE_BUCKET>",
    "arn:aws:s3:::<TF_STATE_BUCKET>/*"
  ]
}

```

```

    },
    {
      "Sid": "TerraformStateLockDynamoDB",
      "Effect": "Allow",
      "Action": [
        "dynamodb:GetItem",
        "dynamodb:PutItem",
        "dynamodb>DeleteItem",
        "dynamodb:UpdateItem",
        "dynamodb:DescribeTable"
      ],
      "Resource": "arn:aws:dynamodb:ap-northeast-2:833453046706:table/<TF_LOCK_TABLE>"
    },
    {
      "Sid": "ECRReadOnly",
      "Effect": "Allow",
      "Action": [
        "ecr:GetAuthorizationToken",
        "ecr:DescribeRepositories",
        "ecr:DescribeImages",
        "ecr:ListImages",
        "ecr:BatchGetImage",
        "ecr:GetDownloadUrlForLayer"
      ],
      "Resource": "*"
    },
    {
      "Sid": "SSMCoreAccess",
      "Effect": "Allow",
      "Action": [
        "ssm:DescribeInstanceInformation",
        "ssm:ListCommandInvocations",
        "ssm:ListCommands",
        "ssm:SendCommand",
        "ssm:GetCommandInvocation",
        "ssm:StartSession",
        "ssm:TerminateSession",
        "ssm:ResumeSession",
        "ssm:GetConnectionStatus"
      ],
      "Resource": "*"
    }
  ]
}

```

3) 고객 관리형 정책 생성

```

aws iam create-policy \
  --policy-name CloudShield-Policy \
  --policy-document file://cloudshield-policy.json

```

4) Role 생성

```
aws iam create-role \
  --role-name CloudShield-Role \
  --assume-role-policy-document file:///github-oidc-trust.json
```

5) 정책 연결

```
aws iam attach-role-policy \
  --role-name CloudShield-Role \
  --policy-arn arn:aws:iam::833453046706:policy/CloudShield-Policy
```

NOTE

OIDC를 사용하면 GitHub Actions에서 장기 Access Key를 저장하지 않아도 된다.

- GitHub가 실행 시점에만 임시 자격 증명을 받아 사용한다.
- 저장소 단위로 Role 사용 범위를 제한할 수 있다.
- CloudShield-Role은 GitHub Actions가 ECR push와 ECS 배포, 일부 인프라 작업을 수행하는 역할이다.

다만 현재 CloudShield-Policy는 ECR push 전용 최소 권한 정책이 아니라, ECS 배포와 일부 인프라 작업까지 포함하는 넓은 권한 정책이다.

따라서 이 설정은 동작상 문제는 없지만, 엄격한 최소 권한 원칙만 놓고 보면 ECR 전용 Role보다 권한 범위가 넓다는 점을 알고 사용해야 한다.

취약한 예시는 다음과 같다.

- GitHub Secrets에 장기 AWS Access Key 저장
- 모든 저장소가 같은 Role을 사용할 수 있게 됨
- Trust Policy에 저장소 제한 없이 OIDC만 연결해 둠
- GitHub Actions에 필요한 범위를 넘는 권한을 부여하고 검토 없이 사용함

체크리스트

- OIDC Provider가 생성되었는가
- CloudShield-Policy가 생성되었는가
- CloudShield-Role이 생성되었는가
- CloudShield-Policy가 CloudShield-Role에 연결되었는가
- Trust Policy에 CloudShield-Lab/CloudShield_Lab 제한이 들어갔는가
- GitHub Secrets에 AWS_ACCOUNT_ID, AWS_REGION, AWS_ROLE_ARN이 등록되었는가

7. CloudWatch Log Group 생성

EC2 애플리케이션 로그를 수집하기 위해 CloudWatch Log Group을 생성할 수 있다.

사용 이름

`/ec2/secure-backend`

7.1 AWS Console 방식

1. AWS Console 접속
2. CloudWatch → Log groups
3. Create log group

4. 아래와 같이 입력

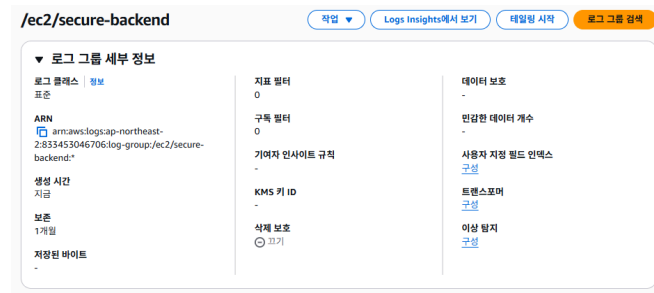
설정

Log group name: /ec2/secure-backend

생성 후 필요하면 로그 보존 기간도 설정한다.

예시

Retention: 30 days



7.2 CLI 방식

```
aws logs create-log-group \
  --log-group-name /ec2/secure-backend \
  --region $AWS_REGION
```

보존 기간 설정 예시

```
aws logs put-retention-policy \
  --log-group-name /ec2/secure-backend \
  --retention-in-days 30 \
  --region $AWS_REGION
```

NOTE

CloudWatch Logs는 EC2에서 실행 중인 애플리케이션 장애나 비정상 접근을 추적하는 데 중요하다.

- 애플리케이션 에러 로그 확인 가능
- 배포 실패 여부 확인 가능
- 추후 CloudWatch Agent 연동 가능

체크리스트

- /ec2/secure-backend 로그 그룹이 생성되었는가
- 보존 기간이 필요에 맞게 설정되었는가
- 추후 CloudWatch Agent 연동 준비가 되었는가

8. EC2 인스턴스 생성

secure 환경용 EC2 인스턴스를 생성한다.

사용 이름

```
Instance: secure-ec2
Type: t3.small
```


OS: Ubuntu
Subnet: secure-subnet-public
Security Group: secure-ec2-sg
Public IP: Enabled
IAM Role: secure-ec2-role

8.1 AWS Console 방식

1) EC2 생성

1. AWS Console 접속
2. EC2 → Instances
3. Launch instances
4. 아래와 같이 입력

설정

Name: secure-ec2
AMI: Ubuntu Server
Instance type: t3.small
VPC: secure-vpc
Subnet: secure-subnet-public
Auto-assign public IP: Enable
Security Group: secure-ec2-sg
IAM instance profile: secure-ec2-role

생성 후 퍼블릭 IP가 할당되었는지 확인한다.

i-0f33a61c078923e37 (secure-ec2)에 대한 인스턴스 요약 정보

less than a minute 전에 업데이트됨

인스턴스 ID i-0f33a61c078923e37	퍼블릭 IPv4 주소 43.202.46.64 개방 주소법	프라이빗 IPv4 주소 10.2.1.200
IPv6 주소 -	인스턴스 상태 실행 중	퍼블릭 DNS ec2-43-202-46-64.ap-northeast-2.compute.amazonaws.com 개방 주소법
호스트 이름 유형 IP 이름: ip-10-2-1-200.ap-northeast-2.compute.internal	프라이빗 IP DNS 이름(IPv4만 해당) ip-10-2-1-200.ap-northeast-2.compute.internal	탄력적 IP 주소 -
프라이빗 리소스 DNS 이름 응답 -	인스턴스 유형 t3.small	AWS Compute Optimizer 찾기 권장 사항을 위해 AWS Compute Optimizer에 검토되었습니다. 자세히 알아보기
자동 할당된 IP 주소 43.202.46.64 [퍼블릭 IP]	VPC ID vpc-060814e7e8c3f716f (secure-vpc)	Auto Scaling 그룹 이름 -
IAM 역할 secure-ec2-role	서브넷 ID subnet-0b79ca6f8d6fb8a3b (secure-subnet-public)	관리형 false
IMDSv2 Required	인스턴스 ARN arn:aws:ec2:ap-northeast-2:833453046706:instance/i-0f33a61c078923e37	
연산자 -		

세부 정보 | 상태 및 경고 | 모니터링 | **보안** | 네트워킹 | 스토리지 | 태그

▼ 보안 세부 정보

IAM 역할 secure-ec2-role	사용자 ID 833453046706	시작 시간 Mon Mar 16 2026 10:41:58 GMT+0900 (한국 표준시)
보안 그룹 sg-0b816946a9763ef5f (secure-ec2-sg)		

2) SSH 접속

```
ssh -i secure-ec2-key.pem ubuntu@<EC2_PUBLIC_IP>
```


3. secure-ec2 선택
4. Connect 또는 SSH를 통해 인스턴스에 접속한다.

2) PostgreSQL 설치 및 서비스 시작

Ubuntu 기준으로 아래 과정을 수행한다.

```
sudo apt update
sudo apt install -y postgresql postgresql-contrib
sudo systemctl start postgresql
sudo systemctl enable postgresql
```

3) DB와 사용자 생성

```
sudo -u postgres psql <<EOF
CREATE USER sentinelshare WITH PASSWORD 'localpassword';
CREATE DATABASE sentinelshare OWNER sentinelshare;
\q
EOF
```

4) 생성 확인

```
sudo -u postgres psql -c "\l"
```

9.2 CLI 방식

Ubuntu 기반 EC2에서 아래처럼 실행한다.

1) 패키지 업데이트

```
sudo apt update
```

2) PostgreSQL 설치

```
sudo apt install -y postgresql postgresql-contrib
```

3) 서비스 시작 및 자동 시작 등록

```
sudo systemctl start postgresql
sudo systemctl enable postgresql
```

4) DB와 사용자 생성

```
sudo -u postgres psql <<EOF
CREATE USER sentinelshare WITH PASSWORD 'localpassword';
CREATE DATABASE sentinelshare OWNER sentinelshare;
\q
EOF
```

5) 생성 확인

```
sudo -u postgres psql -c "\l"
```

NOTE

- 이번 구성은 실습 편의를 위해 PostgreSQL을 secure-ec2에 직접 설치하는 방식이다.
- 더 엄격한 운영 기준에서는 DB를 EC2와 분리해 RDS로 운영하는 것이 더 적절하다.

- DB 비밀번호는 실제 운영에서는 강한 값으로 바꾸고, 가능하면 Secrets Manager를 사용하는 것이 좋다.

체크리스트

- PostgreSQL이 설치되었는가
- postgresql 서비스가 실행 중인가
- sentinelshare 사용자가 생성되었는가
- sentinelshare 데이터베이스가 생성되었는가
- psql -c "\|"에서 데이터베이스가 확인되는가

10. 백엔드 배포 및 실행

secure-ec2에 접속한 뒤 백엔드 코드를 직접 clone하고 실행한다.

1) Git clone

SSH 접속 후 아래를 실행한다.

```
git clone https://github.com/CloudShield-Lab/CloudShield_Lab.git Sentinel_Share
cd Sentinel_Share/sentinel-share-backend
```

2) Node.js 및 의존성 설치

```
sudo apt update
sudo apt install -y nodejs npm git
npm install
```

3) .env 작성

.env 예시

```
NODE_ENV=development
PORT=3000

JWT_SECRET=s3cure123!
JWT_EXPIRES_IN=1h

DB_HOST=localhost
DB_PORT=5432
DB_NAME=sentinelshare
DB_USER=sentinelshare
DB_PASSWORD=localpassword

AWS_REGION=ap-northeast-2
S3_BUCKET_NAME=your-sentinelshare-bucket

PRESIGNED_URL_TTL=300
MAX_FILE_SIZE_MB=100
ALLOWED_MIME_TYPES=image/jpeg,image/png,image/gif,image/webp,application/pdf,text/plain,application/zip,application/x-zip-compressed

CORS_ORIGIN=http://localhost:3000
```

4) 백엔드 실행

```
npm run dev
```

또는

```
node src/app.js
```

```
[ec2-user@ip-10-2-1-199 sentinel-share-backend]$ npm run dev
> sentinel-share-backend@1.0.0 dev
> nodemon src/app.js

[nodemon] 3.1.14
[nodemon] to restart at any time, enter 'rs'
[nodemon] watching path(s): *.*
[nodemon] watching extensions: js,mjs,cjs,json
[nodemon] starting node src/app.js
(node:37421) Warning: NodeDeprecationWarning: The AWS SDK for JavaScript (v3) will no longer support Node.js v18.20.8 in January 2026.
To continue receiving updates to AWS services, bug fixes, and security updates please upgrade to a supported Node.js LTS version.
More information can be found at: https://a.co/c895JFp
(Use 'node-18 --trace-warnings ...' to show where the warning was created)
[SentinelShare] Backend running on port 3000 (production)
```

NOTE

- 현재 코드 기준으로 로컬 PostgreSQL을 사용할 때는 SSL 강제 연결 문제를 피하기 위해 NODE_ENV=development로 두는 것이 안전하다.
- 실제 운영에서는 로컬 DB보다 RDS 분리를 권장한다.
- 장기 운영 시 pm2 또는 systemd로 백엔드 프로세스를 관리하는 것이 좋다.

체크리스트

- Git clone이 완료되었는가
- sentinel-share-backend 폴더에서 npm install이 성공했는가
- .env가 작성되었는가
- npm run dev 실행 시 백엔드가 3000 포트에서 정상 기동하는가
- curl http://localhost:3000/health가 성공하는가

11. 프론트엔드 연결 및 접속 테스트

로컬 PC에서 sentinel-share-frontend를 실행하고 EC2 백엔드와 연결한다.

1) 프론트엔드 env 생성

```
cd Sentinel_Share/sentinel-share-frontend

cat > .env.local <<'EOF'
NEXT_PUBLIC_API_URL=http://<EC2_PUBLIC_IP>:3000
EOF
```

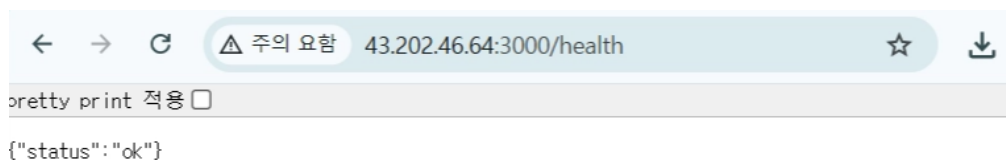
2) 의존성 설치 및 실행

```
npm install
npm run dev
```

3) 백엔드 직접 응답 확인

```
curl http://<EC2_PUBLIC_IP>:3000/health
```

웹브라우저에서 확인



4) CORS 문제 발생 시 확인

백엔드 .env의 CORS_ORIGIN이 프론트 주소와 일치하는지 확인한다.

예시

```
CORS_ORIGIN=http://localhost:3000
```

5) 회원가입 테스트

예시 입력

```
Email: testA@test.com
Password: Test1234
Confirm password: Test1234
```

회원가입 버튼 클릭 후 성공 여부를 확인한다.

NOTE

- 프론트와 백엔드가 모두 3000 포트를 사용하더라도, 호스트가 다르면 충돌하지 않는다.
 - 프론트: http://localhost:3000

- 백엔드: `http://<EC2_PUBLIC_IP>:3000`
- 회원가입 실패 시 가장 먼저 확인할 것은 `CORS_ORIGIN`, `NEXT_PUBLIC_API_URL`, 3000 포트 인바운드 규칙이다.
- Failed to fetch는 브라우저에서 백엔드 요청이 차단되었을 가능성이 높다.

체크리스트

- `sentinel-share-frontend/.env.local`이 생성되었는가
- `NEXT_PUBLIC_API_URL`이 EC2 퍼블릭 IP를 가리키는가
- 프론트엔드가 로컬에서 정상 실행되는가
- `curl http://<EC2_PUBLIC_IP>:3000/health`가 성공하는가
- 회원가입 테스트가 정상적으로 완료되는가

12. Secrets Manager 설정

로컬 PC에서 `sentinel-share-frontend`를 실행하고 EC2 백엔드와 연결한다.

12.1 AWS Console 방식

1) DB 비밀번호 시크릿 생성

1. AWS Console 접속
2. Secrets Manager
3. Store a new secret
4. 아래와 같이 입력

설정

```
Secret type: Other type of secret
Key: password
Value: s3cure123!
Secret name: secure-db-password
```

2) JWT Secret 생성

1. Store a new secret
2. 아래와 같이 입력

설정

```
Secret type: Other type of secret
Key: password
Value: s3cure123!
Secret name: secure-jwt-secret
```

보안 암호

이름, 설명, 태그 키, 태그 값, 보유 서비스 또는 기본 리전을 기준으로 보안 암호 필터링



새 보안 암호 저장

< 1 > ⚙

보안 암호 이름	설명	마지막 검색 날짜(UTC)	생성 날짜(UTC)
secure-db-password	secure-db-password	-	2026년 3월 15일 17시 58분 34초
secure-jwt-secret	secure-jwt-secret	-	2026년 3월 15일 17시 57분 13초

3) Execution Role에 인라인 정책 추가

1. IAM → Roles → s3cure-taskexecutionrole
2. Add permissions → Create inline policy
3. Secrets Manager에 대해 `GetSecretValue` 권한을 허용
4. `secure-db-password`, `secure-jwt-secret`에 접근 가능하도록 제한

▶ 리소스

이러한 작업에 대해 리소스 ARN을 지정했습니다.

```
arn:aws:secretsmanager:ap-northeast-2:833453046706:secret:secure-jwt-secret-*  
arn:aws:secretsmanager:ap-northeast-2:833453046706:secret:secure-db-password-*
```

12.2 CLI 방식

DB 비밀번호 시크릿 생성

```
aws secretsmanager create-secret \  
--name secure-db-password \  
--secret-string '{"password":"s3cure123!"}' \  
--region$AWS_REGION
```

JWT Secret 생성

```
aws secretsmanager create-secret \  
--name secure-jwt-secret \  
--secret-string '{"password":"s3cure123!"}' \  
--region$AWS_REGION
```

인라인 정책 예시 파일 `secrets-inline-policy.json`

```
{  
  "Version": "2012-10-17",  
  "Statement": [  
    {  
      "Sid": "ReadSpecificSecrets",  
      "Effect": "Allow",  
      "Action": [  
        "secretsmanager:GetSecretValue"  
      ],  
      "Resource": [  
        "arn:aws:secretsmanager:ap-northeast-2:AWS_ACCOUNT_ID:secret:secure-db-password*",  
        "arn:aws:secretsmanager:ap-northeast-2:AWS_ACCOUNT_ID:secret:secure-jwt-secret*"  
      ]  
    }  
  ]  
}
```

인라인 정책 추가

```
aws iam put-role-policy \  
--role-name s3cure-taskexecutionrole \  
--policy-name secrets-inline-policy
```



```
--policy-name SecureSecretsReadPolicy \  
--policy-document file:///secrets-inline-policy.json
```

NOTE

Secrets Manager는 비밀번호나 JWT Secret 같은 민감 정보를 코드나 Task Definition에 직접 쓰지 않도록 도와준다.

- 평문 노출 위험 감소
- IAM 기반 접근 제어 가능
- 추후 Secret Rotation 확장 가능

즉, Execution Role이 DB 비밀번호와 JWT secret만 읽을 수 있게 제한한 상태입니다.

체크리스트

- `secure-db-password` 시크릿이 생성되었는가
- `secure-jwt-secret` 시크릿이 생성되었는가
- `s3cure-taskexecutionrole`에 Secret 조회 권한이 추가되었는가
- 권한 범위가 필요한 Secret으로 제한되었는가

13. 자동 배포 및 운영 확인

GitHub Actions와 OIDC를 이용해 안전하게 자동화하고,

배포된 서비스가 ECS, S3, CloudFront 상에서 정상 동작하는지 확인하는 것이 중요하다.

- Attack Dashboard: ECS Fargate 기반 배포
- 사용자 프론트엔드: Next.js 정적 export + S3
- 보안 환경 프론트엔드: CloudFront를 통한 배포
- GitHub Actions 인증 방식: OIDC 기반 AssumeRole

이미 구축한 secure 환경이 자동 배포와 운영 검증까지 연결되어 있는지 확인하는 단계이다.

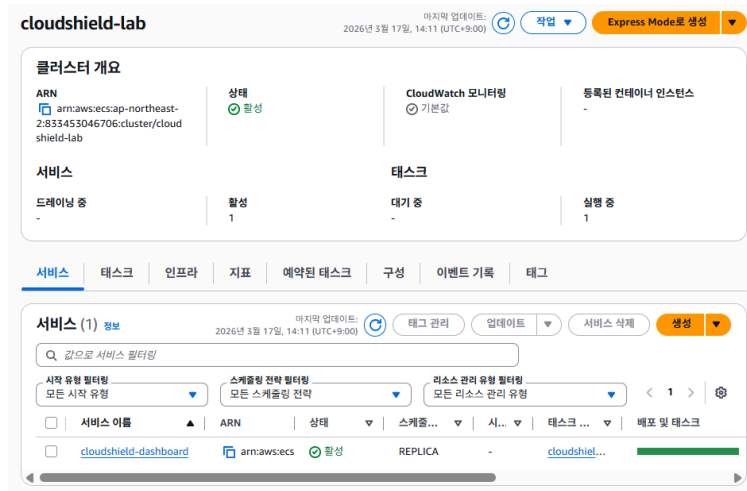
1) Attack Dashboard ECS 서비스 확인

1. AWS Console 접속
2. Amazon ECS → Clusters
3. cloudshield-lab 클러스터 선택
4. **Services** 탭에서 cloudshield-dashboard 서비스 확인
5. 아래 항목을 확인한다

설정

```
Cluster: cloudshield-lab  
Service: cloudshield-dashboard  
Status: 활성  
Running tasks: 1  
Launch type: Fargate
```

서비스가 정상 상태인지, 원하는 수의 태스크가 실행 중인지 확인한다.



2) 대시보드 태스크 정의 확인

1. Amazon ECS → Task definitions
2. cloudshield-dashboard 패밀리 선택
3. 최신 revision 클릭
4. 아래 항목을 확인한다

설정

```
Family: cloudshield-dashboard
Network mode: awsvpc
Requires compatibilities: FARGATE
CPU: 512
Memory: 2048
Container name: cloudshield-dashboard
Container port: 3000
CloudWatch Log Group: /ecs/cloudshield-dashboard
```

또한 환경 변수와 시크릿이 정상 연결되어 있는지 확인한다.

예시

```
AWS_REGION=ap-northeast-2
NODE_ENV=production

Secrets:
VULNERABLE_API_URL
VULNERABLE_S3_BUCKET
AWS_API_URL
AWS_S3_BUCKET
```

이 설정은 대시보드가 ECS Fargate에서 실행되며, CloudWatch Logs와 외부 설정을 함께 사용하는지 확인하는 절차이다.

3) 보안 환경 프론트엔드 S3 / CloudFront 구성 확인

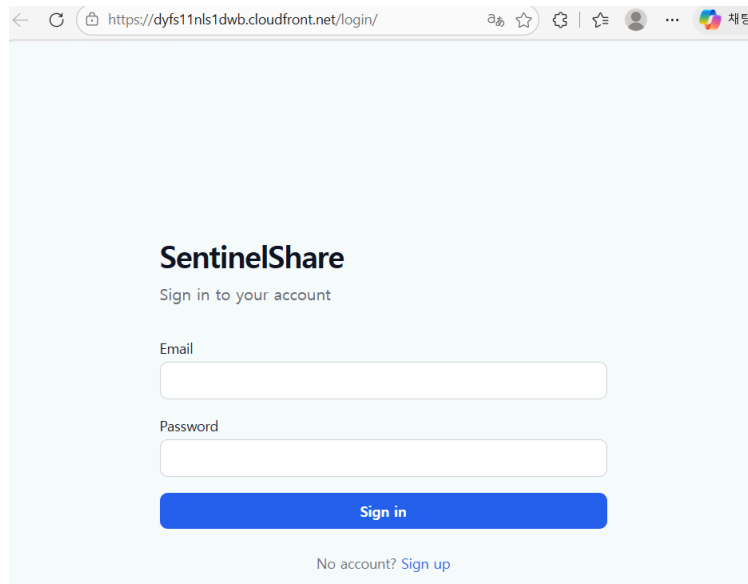
1. AWS Console 접속
2. Amazon S3 → sentinel-share-secure-frontend 버킷 선택

3. 버킷 안에 정적 프론트엔드 파일이 업로드되어 있는지 확인한다
4. Amazon CloudFront → 배포 목록으로 이동한다
5. sentinel-share-secure-frontend를 오리진으로 사용하는 배포가 존재하는지 확인한다
6. CloudFront 도메인으로 실제 접속이 되는지 확인한다

예시

<https://dyfs11nls1dwb.cloudfront.net/signup/>

즉 보안 환경 프론트엔드는 S3에 정적 파일을 저장하되, 사용자 접근은 CloudFront를 통해 이루어지는 구조임을 확인하는 단계이다.



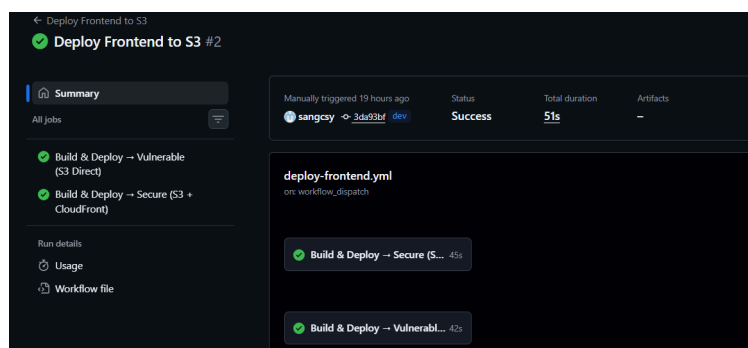
4) GitHub Actions 프론트엔드 자동 배포 확인

1. GitHub 저장소 접속
2. **Actions** 탭으로 이동
3. Deploy Frontend to S3 워크플로 선택
4. 가장 최근 실행을 클릭한다
5. 아래 job이 모두 성공했는지 확인한다

확인 항목

Build & Deploy → Secure (S3 + CloudFront)

보안 환경 프론트엔드가 GitHub Actions를 통해 자동 배포되고 있는지 확인



5) GitHub Actions 대시보드 자동 배포 확인

1. GitHub 저장소 접속
2. **Actions** 탭으로 이동
3. Deploy Attack Dashboard to ECS 워크플로 선택
4. 가장 최근 실행을 클릭한다
5. 최근 배포가 성공했는지 확인한다
6. 필요하면 job 상세에서 이미지 빌드 및 ECS 배포 단계가 모두 성공했는지 확인한다

6) GitHub Secrets 및 OIDC Role 연결 확인

1. GitHub 저장소 접속
2. **Settings** → **Secrets and variables** → **Actions** 이동
3. 아래 Secret이 등록되어 있는지 확인한다

확인 항목

```
AWS_ROLE_ARN
DASHBOARD_ECS_CLUSTER
DASHBOARD_ECS_SERVICE
```

1. AWS Console → IAM → Roles 이동
2. CloudShield-Role 선택
3. **신뢰 관계(Trust relationships)** 탭에서 아래 항목이 포함되어 있는지 확인한다

확인 포인트 예시

```
token.actions.githubusercontent.com
sts:AssumeRoleWithWebIdentity
repo:CloudShield-Lab/CloudShield_Lab:*
```

즉 GitHub Actions는 장기 Access Key 대신 CloudShield-Role을 OIDC 방식으로 Assume 하여 AWS 배포를 수행한다.

7) 배포 구성 파일 확인

대시보드 자동 배포 워크플로 파일

```
.github/workflows/deploy-dashboard.yml
```

핵심 내용

```
name: Deploy Attack Dashboard to ECS
branches: [master, dev]
ECR_REPOSITORY: cloudshield-dashboard
role-to-assume: ${ secrets.AWS_ROLE_ARN }
task-definition: attack-dashboard/infra/ecs-task-definition-dashboard.json
service: ${ secrets.DASHBOARD_ECS_SERVICE }
cluster: ${ secrets.DASHBOARD_ECS_CLUSTER }
```

프론트엔드 자동 배포 워크플로 파일

```
.github/workflows/deploy-frontend.yml
```

핵심 내용 예시

Deploy Frontend to S3
Build & Deploy → Secure (S3 + CloudFront)

대시보드 태스크 정의 파일

attack-dashboard/infra/ecs-task-definition-dashboard.json

핵심 내용 예시

```
"family": "cloudshield-dashboard"
"networkMode": "awsvpc"
"requiresCompatibilities": ["FARGATE"]
"cpu": "512"
"memory": "2048"
"name": "cloudshield-dashboard"
"containerPort": 3000
"hostPort": 3000
"awslogs-group": "/ecs/cloudshield-dashboard"
```

운영 자동화는 GitHub Actions 워크플로와 ECS 태스크 정의 파일을 기준으로 동작한다.

NOTE

이 단계의 핵심은 secure 환경을 단순히 만드는 것에서 끝내지 않고, 안전한 배포 방식과 운영 검증 체계까지 연결하는 것이다.

- Attack Dashboard는 ECS Fargate 표준 배포 방식을 사용하므로 배포와 롤백 관리가 명확하다.
- 보안 환경 프론트엔드는 S3 + CloudFront 구조를 사용하여 직접 S3 노출보다 안전한 접근 경로를 가진다.
- GitHub Actions는 장기 AWS Access Key를 저장하지 않고, OIDC 기반 AssumeRole로 인증한다.
- 현재 OIDC용 역할은 CloudShield-Role이며, ECS 실행 역할과 구분해서 이해해야 한다.

체크리스트

- cloudshield-lab ECS 클러스터가 존재하는가
- cloudshield-dashboard 서비스가 활성 상태인가
- 대시보드 태스크가 FARGATE 기반으로 실행 중인가
- 대시보드 컨테이너가 포트 3000으로 설정되었는가
- CloudWatch 로그 그룹 /ecs/cloudshield-dashboard가 연결되었는가
- sentinel-share-secure-frontend 버킷이 존재하는가
- 보안 환경 프론트엔드가 S3에 정상 배포되었는가
- 보안 환경 프론트엔드가 CloudFront를 통해 정상 접속되는가
- Deploy Frontend to S3 워크플로의 Secure (S3 + CloudFront) job이 성공했는가
- Deploy Attack Dashboard to ECS 워크플로가 성공했는가
- GitHub Actions가 CloudShield-Role을 OIDC 방식으로 Assume 하도록 구성되었는가
- AWS_ROLE_ARN, DASHBOARD_ECS_CLUSTER, DASHBOARD_ECS_SERVICE가 GitHub Secrets에 등록되었는가